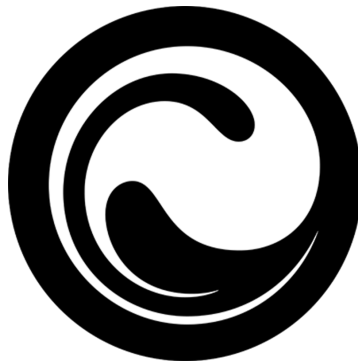


Enso CCIP Receiver

Smart Contract Security Assessment

January 09, 2026



ABSTRACT

Dedaub was commissioned to perform a security audit of Enso CCIP Receiver Protocol, which builds on top of Chainlink's CCIP. Only a minor issue was identified which was properly addressed.

BACKGROUND

The Enso CCIP Receiver is a destination-side contract that integrates Enso Shortcuts with Chainlink's Cross-Chain Interoperability Protocol (CCIP). It serves as a bridge endpoint that receives cross-chain messages containing ERC-20 tokens and executes Enso Shortcuts operations on the destination chain.

Enso Shortcuts is a composable DeFi routing system that enables complex multi-protocol operations in a single transaction. The CCIP Receiver allows users to initiate Shortcuts operations cross-chain by:

1. Sending tokens via CCIP from a source chain
2. Receiving tokens and execution data on the destination chain
3. Automatically routing tokens through Enso Shortcuts to execute complex DeFi operations

SETTING & CAVEATS

This audit report mainly covers the contracts of the [EnsoBuild/shortcuts-client-contracts](#) repository of the Enso CCIP Receiver Protocol at commit [c376312a9e43c3792aa29034be247f4f6de6f273](#) of [PR #60](#).

As part of the audit we also reviewed the fixes of the issues included in the report, which were implemented as new commits of [PR #60](#) up to commit [8b3b141d70847be380762f0991292a011f7a1353](#).

Audit Start Date: January 08, 2026

Report Submission Date: January 09, 2026

Two auditors worked on the following contracts:

```
src/
├── bridge/
│   └── EnsoCCIPReceiver.sol
├── interfaces/
│   ├── IEnsoCCIPReceiver.sol
│   └── ITypeAndVersion.sol
└── libraries/
    └── CCIPMessageDecoder.sol

script/
└── EnsoCCIPReceiverDeployer.s.sol
```

The audit's main target is security threats, i.e., what the community understanding would likely call "hacking", rather than the regular use of the protocol. Functional correctness (i.e. issues in "regular use") is a secondary consideration. Typically it can only be covered if we are provided with unambiguous (i.e. full-detail) specifications of what is the expected, correct behavior. In terms of functional correctness, we often trusted the code's calculations and interactions, in the absence of any other specification. Functional correctness relative to low-level calculations (including units, scaling and quantities returned from external protocols) is generally most effectively done through thorough testing rather than human auditing.

PROTOCOL-LEVEL CONSIDERATIONS

ID	Description	STATUS
P1	Consideration for expansion to non-EVM chains	INFO
Currently, the protocol operates over various EVM chains. However, if the team decides to expand to non-EVM chains like Solana or Sui in the future, there is a subtlety regarding CCIP that should be taken into account, specifically regarding the token and message receivers. In short, the two addresses are decoupled on these networks, which allows sending tokens to addresses not controlled by the message receiver. If the latter does not verify its token balances or does not perform other sanity checks, it could end up spending its own reserves. Sending tokens from EVM to Solana, for example, requires defining the token receiver separately using the <code>EVM2AnyMessage.extraArgs</code> encoding a <code>SVMExtraArgsV1</code> struct. @chainlink-ccip/chains/evm/.../Client.sol::SVMExtraArgsV1:L68 <pre>struct SVMExtraArgsV1 { uint32 computeUnits; uint64 accountIsWritableBitmap; bool allowOutOfOrderExecution; bytes32 tokenReceiver; bytes32[] accounts; }</pre>		

This is required because on Solana, the tokens of a particular receiver are sent to a unique [Token Account](#) with an address derived by the receiver's and the mint account's addresses (ref. [Associated Token Account \(ATA\)](#)).

Thus, the message and the token receiver addresses have to be defined separately. CCIP does not enforce the inherent connection between the two, which means that a sender can define any random token receiver other than the message receiver's Token Account address when sending a message with tokens to Solana. Other bridging paths, like sending to EVM chains, do not have this nuance since the token receiver is enforced to be the same as the message receiver.

Call stack for messages originating from EVM chains:

```
"@chainlink-ccip/evm/FeeQuoter.sol::_processChainFamilySelector:1206"  
  ^ (called by FeeQuoter.sol::processMessageArgs)  
  ^ (called by OnRamp.sol::forwardFromRouter)  
  
function _processChainFamilySelector(  
    uint64 destChainSelector,  
    bytes calldata messageReceiver,  
    bytes calldata extraArgs  
) internal view returns (  
    bytes memory validatedExtraArgs, ..., bytes memory tokenReceiver  
) {  
    ...  
    if (  
        destChainConfig.chainFamilySelector ==  
            Internal.CHAIN_FAMILY_SELECTOR_EVM  
        || destChainConfig.chainFamilySelector ==  
            Internal.CHAIN_FAMILY_SELECTOR_APTOS  
        || destChainConfig.chainFamilySelector ==
```

```
        Internal.CHAIN_FAMILY_SELECTOR_TVM
    ) {
        Client.GenericExtraArgsV2 memory parsedExtraArgs =
            _parseUnvalidatedEVMEExtraArgsFromBytes(
                extraArgs, destChainConfig.defaultTxGasLimit);

        // Dedaub:
        // For EVM destination chains, the messageReceiver is returned
        // as the tokenReceiver. For Sui and Solana, a different address
        // is returned, extracted from extraArgs.

        return (Client._argsToBytes(parsedExtraArgs),
            parsedExtraArgs.allowOutOfOrderExecution, messageReceiver);
    }
    if (destChainConfig.chainFamilySelector ==
        Internal.CHAIN_FAMILY_SELECTOR_SUI) {
        ...
        return (
            extraArgs,
            true,
            abi.encode(
                _parseSuiExtraArgsFromBytes(
                    extraArgs, destChainConfig.maxPerMsgGasLimit,
                    destChainConfig.enforceOutOfOrder
                ).tokenReceiver
            )
        );
    }
    if (destChainConfig.chainFamilySelector ==
        Internal.CHAIN_FAMILY_SELECTOR_SVM) {
        ...
    }
```

```

    return (
        extraArgs,
        true,
        abi.encode(
            _parseSVMExtraArgsFromBytes(
                extraArgs, destChainConfig.maxPerMsgGasLimit,
                destChainConfig.enforceOutOfOrder
            ).tokenReceiver
        )
    );
}
revert InvalidChainFamilySelector(destChainConfig.chainFamilySelector);
}

```

"@chainlink-ccip/evm/OnRamp.sol::forwardFromRouter:246"

(called by Router.sol::ccipSend)

```

function forwardFromRouter(
    ..., Client.EVM2AnyMessage calldata message, ...
) external returns (bytes32) {

    Internal.EVM2AnyRampMessage memory newMessage =
        Internal.EVM2AnyRampMessage({
            ...
            extraArgs: "",
            receiver: message.receiver,
            ...
        });
    ...
    bytes memory tokenReceiver;
}

```

```
(newMessage.feeValueJuels, isOutOfOrderExecution,  
    newMessage.extraArgs, tokenReceiver  
) = IFeeQuoter(  
    s_dynamicConfig.feeQuoter  
).processMessageArgs(destChainSelector, message.feeToken,  
feeTokenAmount,  
    message.extraArgs, message.receiver);  
    ...  
}
```

(ref. [chainlink-ccip/OnRamp.sol#L246](#), [chainlink-ccip/FeeQuoter.sol#L1206](#))

As a result, this difference requires attention from the external protocols that build on top of CCIP on non-EVM chains, to ensure that the tokens bridged are received at the right address before executing the incoming message. Otherwise, an adversary could bridge the tokens to an address of their control and have the message receiver, i.e., the protocol, run using its own reserves.

“Relevant note from Chainlink’s official docs: [CCIP Best Practices \(SVM\)](#)”

VULNERABILITIES & FUNCTIONAL ISSUES

This section details issues affecting the functionality of the contract. Dedaub generally categorizes issues according to the following severities, but may also take other considerations into account such as impact or difficulty in exploitation:

Category	Description
CRITICAL	Can be profitably exploited by any knowledgeable third-party attacker to drain a portion of the system's or users' funds OR the contract does not function as intended and severe loss of funds may result.
HIGH	Third-party attackers or faulty functionality may block the system or cause the system or users to lose funds. Important system invariants can be violated.
MEDIUM	Examples: • User or system funds can be lost when third-party systems misbehave. • DoS, under specific conditions. • Part of the functionality becomes unusable due to a programming error.
LOW	Examples: • Breaking important system invariants but without apparent consequences. • Buggy functionality for trusted users where a workaround exists. • Security issues which may manifest when the system evolves.

Issue resolution includes “dismissed” or “acknowledged” but no action taken, by the client, or “resolved”, per the auditors.

CRITICAL SEVERITY:

[No critical severity issues]

HIGH SEVERITY:

[No high severity issues]

MEDIUM SEVERITY:

[No medium severity issues]

LOW SEVERITY:

ID	Description	STATUS
L1	<code>_tryDecodeMessageData</code> succeeds when superfluous bytes exists in the input	RESOLVED
<code>CCIPMessageDecoder.sol::_tryDecodeMessageData</code> decodes a bytestream of the form <code>abi.encode(address,bytes)</code>. The function properly decodes the message and returns false when the length of the input is smaller than the byte array. However, if the length of the input is larger, then the extra bytes are simply ignored. <code>CCIPMessageDecoder.sol::_tryDecodeMessageData:55</code> <pre>function _tryDecodeMessageData(bytes memory _data) internal pure returns (...) { ... unchecked {</pre>		

```

...
// Require len itself to fit in the available tail
if (len > avail) {
    return (false, address(0), bytes(""));
}

// Ceil32(len) and ensure padded bytes also fit
// (defensive; usually implied by len<=avail)
uint256 padded = (len + 31) & ~uint256(31);
if (padded > avail) {
    return (false, address(0), bytes(""));
}
...
}
return (true, receiver, shortcutData);
}

```

Although having superfluous bytes does not impose an immediate problem, it would be good practice to reject such input, as it might indicate some bug or unwanted condition in other parts of the code. It is generally standard practice to ensure that such streams are fully consumed (see, for instance, [this example](#)).

L2

Outdated assumption for CCIP token transfers

ACKNOWLEDGED

The protocol assumes that CCIP only supports sending **1** token per message, as stated in a relevant comment in `EnsoCCIPReceiver.sol`.

`EnsoCCIPReceiver.sol::_validateMessage:198`

```

function _validateMessage(
    Client.Any2EVMMessage memory _message

```

```
) private view returns (...) {  
    ...  
    if (destTokenAmounts.length > 1) {  
        // CCIP currently delivers at most ONE token per message.  
        // Multiple-token deliveries are not supported by the  
        // protocol today, so treat any length > 1 as invalid and  
        // quarantine/refuse.  
        return (token, amount, receiver, shortcutData,  
            ErrorCode.TOO_MANY_TOKENS, errorData);  
    }  
    ...  
}
```

Chainlink, though, has implemented CCIP so that it supports multiple token transfers for the EVM chains in both of the most recently used versions ([v1.5.0](#), [v1.6.0](#)). However, there is also the `maxNumberOfTokensPerMsg` variable, which dynamically controls the allowance of token transfers per message ([v1.5.0](#), [v1.6.0](#)). For v1.5.0, indeed, most of the lanes have been set to allow only 1 token transfer, which could justify the assumption mentioned above, but this seems to be changing as Chainlink migrates its lanes to v1.6.0.

For example, from the list of supported chains found in the protocol's deployment script, **Monad** (chain selector `8481857512324358265`) and **Plasma** (chain selector `9335212494177455608`) now use v1.6.0 and **Plasma** has been updated to allow up to **10** concurrent token transfers per message according to the [current on-chain state](#):

5. getDestChainConfig (0x6def4ce7)

destChainSelector (uint64)

9335212494177455608

Query

↳ tuple

[getDestChainConfig(uint64) method Response]

» tuple : [true,"10","30000","3000000","300000","16","40","3000","100","16","1","0x2812d52c",false,"25","90000","200000","11000000000000000000","0","10"]

@chainlink-ccip -> FeeQuoter.sol::DestChainConfig:105

```
struct DestChainConfig {
    bool isEnabled;
    uint16 maxNumberOfTokensPerMsg;
    uint32 maxDataBytes;
    uint32 maxPerMsgGasLimit;
    uint32 destGasOverhead;
    ...
}
```

As a result, you may want to reconsider the current assumption and adjust the code accordingly to be compatible with the new version of CCIP and any future updates.

Note:

“This is the [OnRamp](#) of v1.6.0 that points to this [FeeQuoter](#), which stores the [DestChainConfig](#) mapping with all the new configurations per lane. You can query it by using the above or any supported chain selector that can be found in Chainlink’s official [CCIP directory](#).

For v1.5.0, this data lived in individual *OnRamps* since each lane had a unique deployment instead of the unified and shared one used in v1.6.0. Hence, you can find the same information by querying their *DynamicConfig* data (i.e. for the [BNB chain](#))."

OTHER / ADVISORY ISSUES:

This section details issues that are not thought to directly affect the functionality of the project, but we recommend considering them.

ID	Description	STATUS
A1	Redundant error check	RESOLVED

In `EnsoCCIPReceiver.sol::_getRefundPolicy`, the `NO_ERROR` condition will never be true, as this function is only called by `_ccipReceive` when the `errorCode` is different from `NO_ERROR`. Of course, it may be there for completeness, but we mention it for visibility.

`EnsoCCIPReceiver.sol::_ccipReceive:74`

```
function _ccipReceive(
    Client.Any2EVMMessage memory _message
) internal override {
    ...
    if (errorCode != ErrorCode.NO_ERROR) {
        ...
        RefundKind refundKind = _getRefundPolicy(errorCode);
        ...
    }
    ...
}
```

EnsoCCIPReceiver.sol::_ccipReceive:74

```

function _getRefundPolicy(
    ErrorCode _errorCode
) private pure returns (RefundKind) {

    // Dedaub:
    // _errorCode == ErrorCode.NO_ERROR will never be true since the same
    // check is performed before calling this function.

    if (_errorCode == ErrorCode.NO_ERROR ||
        _errorCode == ErrorCode.ALREADY_EXECUTED) {
        return RefundKind.NONE;
    }
    ...
}

```

A2

Compiler bugs

INFO

The code has a floating Solidity `^0.8.24` version. For deployment, we recommend no floating pragmas but a specific version to be confident about the baseline guarantees offered by the compiler. Version `0.8.24`, in particular, has no [known bugs](#).

DISCLAIMER

The audited contracts have been analyzed using automated techniques and extensive human inspection in accordance with state-of-the-art practices as of the date of this report. The audit makes no statements or warranties on the security of the code. On its own, it cannot be considered a sufficient assessment of the correctness of the contract. While we have conducted an analysis to the best of our ability, it is our recommendation for high-value contracts to commission several independent audits, a public bug bounty program, as well as continuous security auditing and monitoring through Dedaub Security Suite.

ABOUT DEDAUB

Dedaub offers significant security expertise combined with cutting-edge program analysis technology to secure some of the most prominent protocols in DeFi. The founders, as well as many of Dedaub's auditors, have a strong academic research background together with a real-world hacker mentality to secure code. Protocol blockchain developers hire us for our foundational analysis tools and deep expertise in program analysis, reverse engineering, DeFi exploits, cryptography and financial mathematics.